CIS*3110 Operating Systems

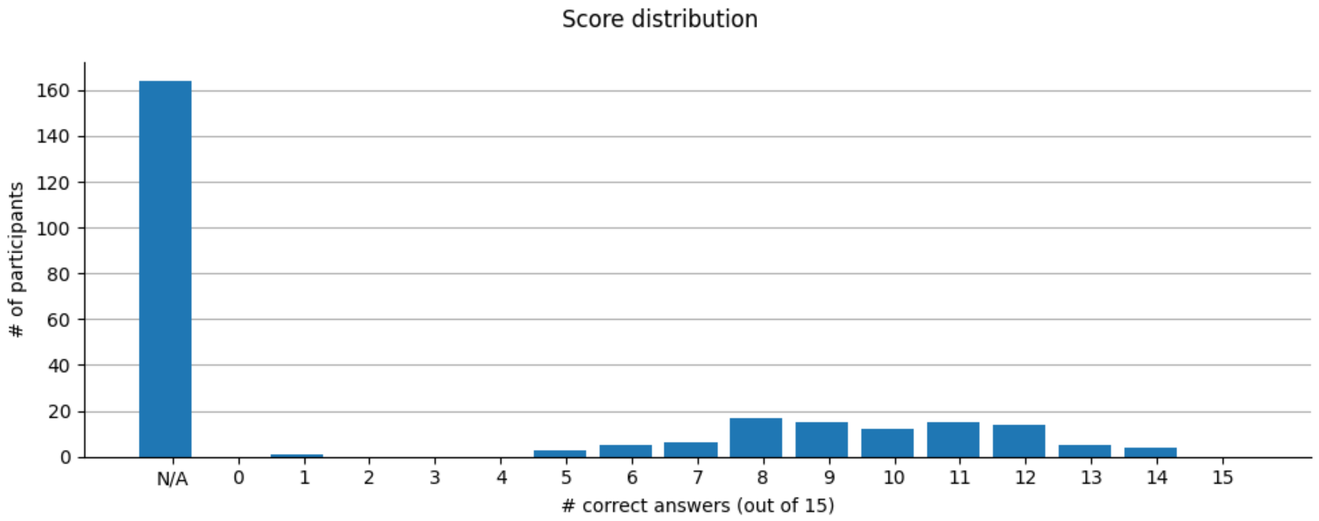
Week 8
Lecture #12 Midterm review

Prof. Yulia Kotseruba
February 24, 2026

Quiz 10

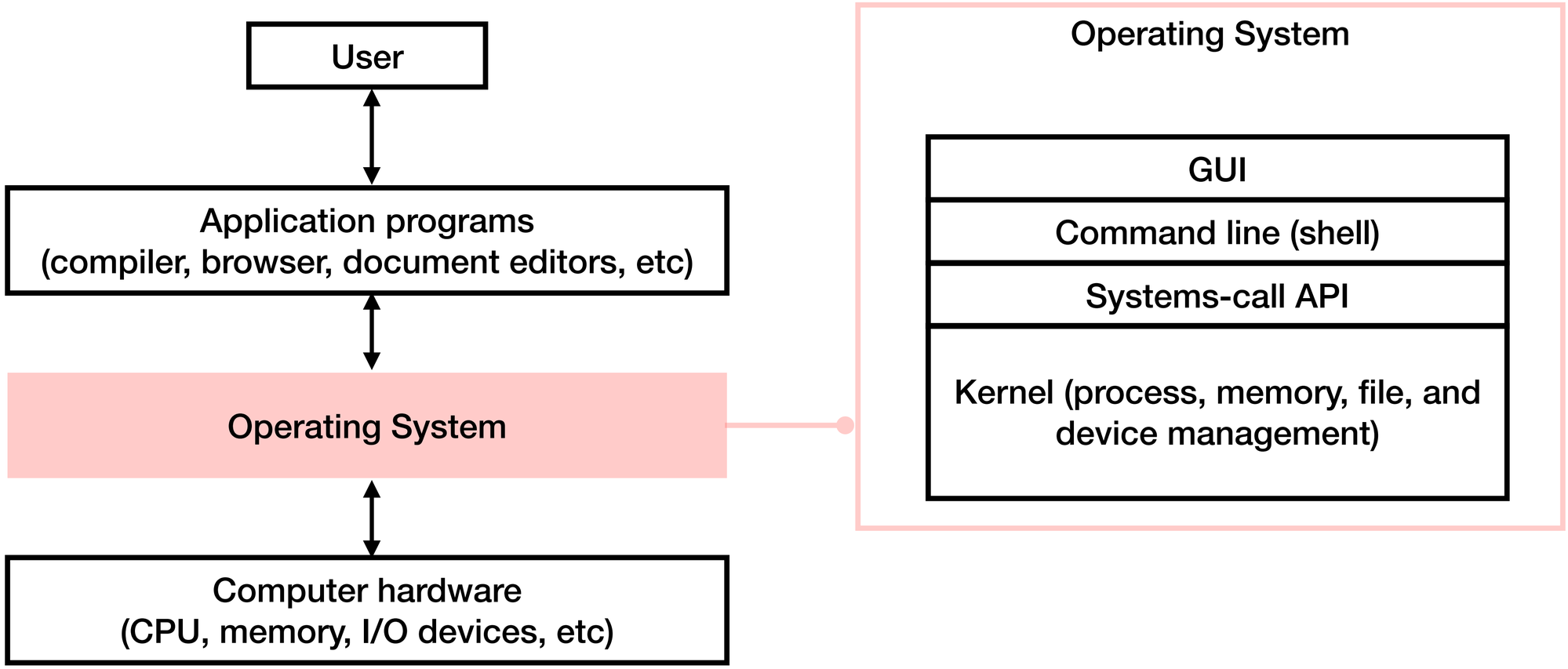
```
int global_var = 1;
int main(void) {
        if (fork()==0) global_var++;
        if (fork()==0) {
                global_var++;
        }
}
```

- **56%** average (down from 57% in quiz 8)
- 3 hardest questions:
  - (41%) Assuming a 16-bit virtual address space and 1KB pages, how many entries will the page table contain? a) 10 b) 16 c) 32 d) 64
  - (45%) When combining paging and segmentation, how many page tables are needed per process? a) 1 b) 2 c) 3 d) 4
  - (45%) What is the max possible value of global_var when this code executes? a) 1 b) 2 c) 3 d) 4



What is an operating system?

- Intermediary component between the users and the computer hardware



Operating Systems facts

- Operating system is a **middleware** between users and their applications and computer hardware
  - Provides a text and graphical UI for users and system call API for user applications
- Most of the **1000+** OSs today descend from a handful of projects in 1950-1980s
- Unix is one of the most influential OSs, developed in the 1970s
- C (designed for **UNIX**) remains the main **systems programming** language

Operating Systems goals

- Goal #1 convenience and needs of users running programs
- Goal #2 use the hardware in the most efficient way possible
- Goal #3 ensure correctness of the programs and prevent interference

- OS **manages** all aspects of running a program
  - taking user commands via UI, loading program into memory, execution of the program, handling exceptions, managing resources, dealing with I/O, returning the results back to the user

Operating Systems techniques

- Three general techniques that most OSs use to achieve these goals are:
  - **Virtualization** of CPU and memory — sharing compute and memory across many processes
  - **Concurrency** — ensuring correctness during parallel execution of processes that share resources
  - **Persistence** — managing permanently stored information

Process state and memory

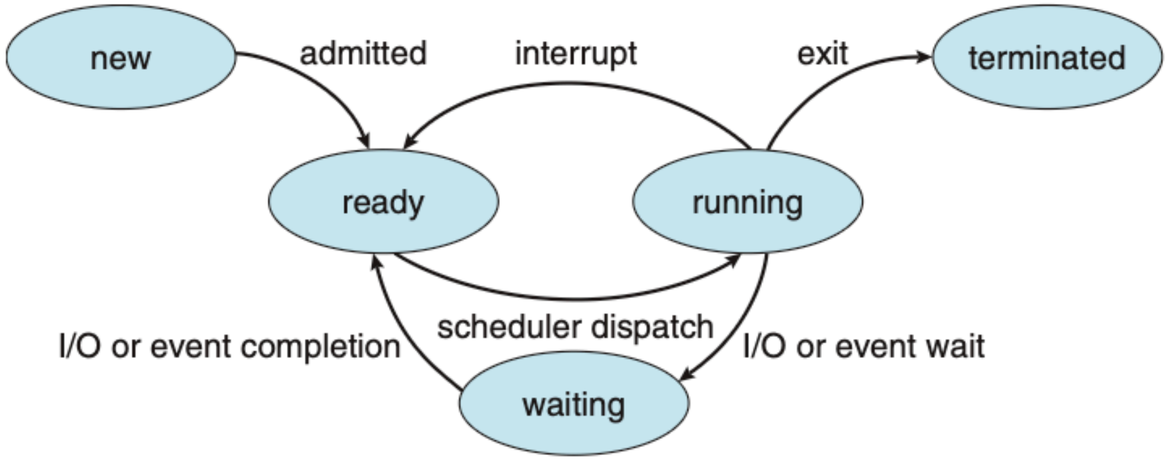
- Processes are **running programs** that change their state
- Processes have a **virtual memory**
  - pros: simplifies compilation, illusion of larger memory than available
  - cons: need to translate virtual addresses to physical to access data, danger of trespassing memory of other processes/OS
- OS **monitors** and **controls** the state of all processes
- OS **allocates** and **manages** virtual/physical memory for all processes

Process data: PCB

- **Process control block (PCB)** is the representation of the process in an OS
  - **Process state** (new, ready, blocked…)
  - **CPU registers** - program counter, stack pointer, function parameters,…
  - **Address space** - memory layout
  - **CPU scheduling** - process priority, scheduling queues
  - **I/O status** - e.g. a list of open files, any devices that the program interacts with
  - and more

Process state

- **new -> ready**
  - new process waits for the scheduler
- **ready -> running**
  - CPU begins or resumes the execution
- **running -> terminated**
  - program completes the task, is terminated by its parent, terminated by the OS (e.g. due to invalid instruction, overflow, memory protection violation)
- **running -> ready**
  - scheduler preempted the process to run a different process
- **running -> waiting**
  - process made a system call to wait for I/O operation, resource, sleep, message from another process
- **waiting -> ready**
  - I/O done, process goes to the scheduler's queue



state diagram of process states

Figure 3.2 from Silberchatz et al 10ed.

Process API

- OS provides system calls for process management
  - **Create** - creates a new process (e.g. when you run a program from UI)
  - **Wait** - process waits for something to happen
  - Various **control** functions - calls for suspending/resuming the process
  - **Status** - call for checking on the process (e.g. uptime, state)
  - **Destroy** - may be invoked before or after the program is finished running

Process API: UNIX

- `fork()` - create a new process by duplicating the calling process
- `exec()` - transform currently running process into a different process
- `wait()` - halt execution of a process until a condition is satisfied
- `kill()` - terminate a process

Process API: fork()

- **fork()** - creates a **new process** (child) *almost* identical to the calling process (parent)
  - child has a **copy** of the **parent's memory** address space, heap and stack
  - fork() **returns twice**: 0 to a child, child's pid to the parent, -1 if failed
  - child has a **unique PID**, different from the parent
  - child has a **copy** of the parent's **file descriptors**
  - even if the parent process is multithreaded, the child has only **1 thread**
  - concurrency stuff is **not inherited** (locks, signals, alarms)

Process API: exec()

- exec() - **does not create a new process**, it **replaces** the program in the current process with another program
  - **stops** the current process
  - replaces the **entire contents** of the current process
    - **pid of the process stays the same** but memory is reinitialized
  - **loads the new program** into the process space
  - **runs** that program

Process API: kill()

- `kill` - sends a signal to the process or a group of processes with specified pid(s)
  - `SIGTERM` - standard signal to terminate a process
  - SIGINT - interrupt from keyboard (Ctrl+C)
  - `SIGKILL` - force terminate immediately, all resources are released right away
  - SIGSTOP - process stops, program continues to hold its resources, doesn't use the CPU
  - **SIGKILL** and **SIGSTOP** cannot be caught, blocked or ignored
- Works via systems call `kill(pid, sig)` or by typing `kill <SIG> <PID>` in terminal
  - `kill -9 1234` will immediately terminate process 1234
  - -9 stands for SIGKILL
- run `man 3 signal` to see what signals are available on your terminal

CPU scheduling

- **Non-preemptive scheduling**
  - Jobs run until they are finished or release the CPU themselves (e.g. call sleep())
  - Does not need a PCB to monitor state
- **Preemptive scheduling**
  - Jobs can be interrupted at any time
  - PCB is needed - tracking status of the processes, context switching
  - Timer interrupts are needed to make scheduling decisions

CPU scheduling

- **Non-preemptive**
  - FIFO, SJF - simple, impractical
- **Preemptive**
  - PSJF, RR - simple, low overhead
  - MLFQ - the most general, many parameters, hard to tune
  - Lottery, Stride - simple, low overhead, fair-share, but hard to set parameters (tickets/stride)
  - FCS, EEVDF - complex, low overhead, fair-share, no heuristics
    - main innovations: variable CPU share depending on the priority + red-black trees for efficiency

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

PSJF when a tie arises, avoid incurring a context switch

A																								

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

PSJF when a tie arises, avoid incurring a context switch

A	B																							

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

A is preempted because B arrives and is shorter

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

PSJF when a tie arises, avoid incurring a context switch

A	B	C	C																					

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

B is preempted because C arrives and is shorter

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

PSJF when a tie arises, avoid incurring a context switch

A	B	C	C																					

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

C is **not** preempted because it has the same time as D and we don't want to incur a context switch

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

PSJF when a tie arises, avoid incurring a context switch

A	B	C	C	D																				

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

D runs next, it arrived while C was running and has the shortest running time so far

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

PSJF when a tie arises, avoid incurring a context switch

A	B	C	C	D																				

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

D is done

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

PSJF when a tie arises, avoid incurring a context switch

A	B	C	C	D	B	B	B																	

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

B is the shortest, runs next

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

PSJF when a tie arises, avoid incurring a context switch

A	B	C	C	D	B	B	B																	

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

B is done

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

PSJF when a tie arises, avoid incurring a context switch

A	B	C	C	D	B	B	B	A	A	A	A	A	A	A	E	E	E	E	E	E	E	E	E	E

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

A runs next because it has less CPU time than E

E runs last

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

RR (2) when a tie arises, avoid incurring a context switch

A	A																							

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

RR (2) when a tie arises, avoid incurring a context switch

A	A	B	B																					

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

RR (2) when a tie arises, avoid incurring a context switch

A	A	B	B	A	A																			

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

A was reinserted into the queue at t=2, then C was inserted

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

RR (2) when a tie arises, avoid incurring a context switch

A	A	B	B	A	A	C	C																	

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

RR (2) when a tie arises, avoid incurring a context switch

A	A	B	B	A	A	C	C	D																

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

D runs after C

D arrived at 3 and was inserted after A and C

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

RR (2) when a tie arises, avoid incurring a context switch

A	A	B	B	A	A	C	C	D																

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

D is done

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

RR (2) when a tie arises, avoid incurring a context switch

A	A	B	B	A	A	C	C	D	B	B														

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

B finished its quantum just as E arrived at t=4

to avoid context switch, we reinserted B before adding a new process E

B runs next

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

RR (2) when a tie arises, avoid incurring a context switch

A	A	B	B	A	A	C	C	D	B	B	E	E												

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

B is done, E runs after B

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

RR (2) when a tie arises, avoid incurring a context switch

A	A	B	B	A	A	C	C	D	B	B	E	E	A	A	E	E	A	A	E	E	E	E	E	E

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

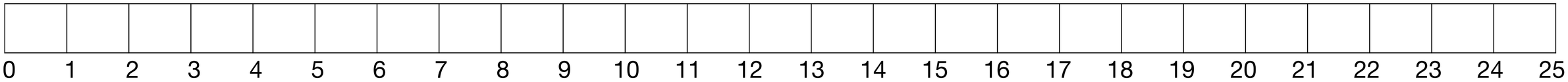
A and E finish their CPU bursts in a regular round robin

CPU scheduling: practice

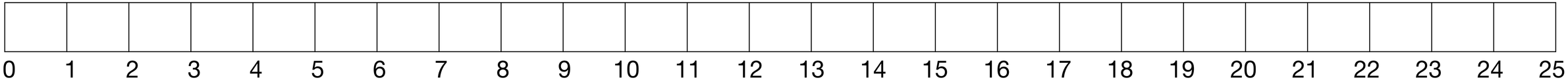
PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

MLFQ when a tie arises, avoid incurring a context switch

q=3
a=2



q=5

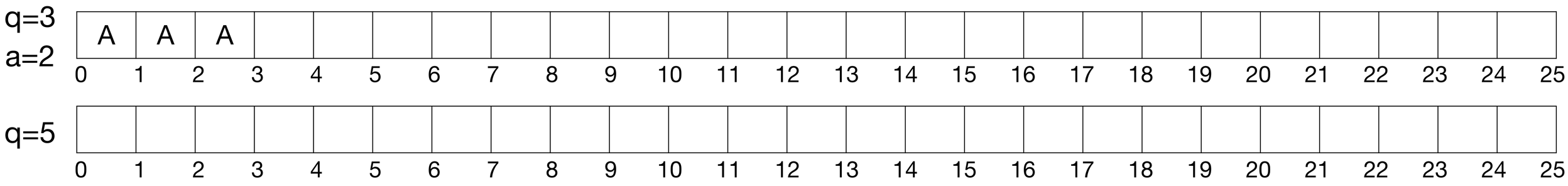


CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

MLFQ when a tie arises, avoid incurring a context switch

processes always start in the top priority queue

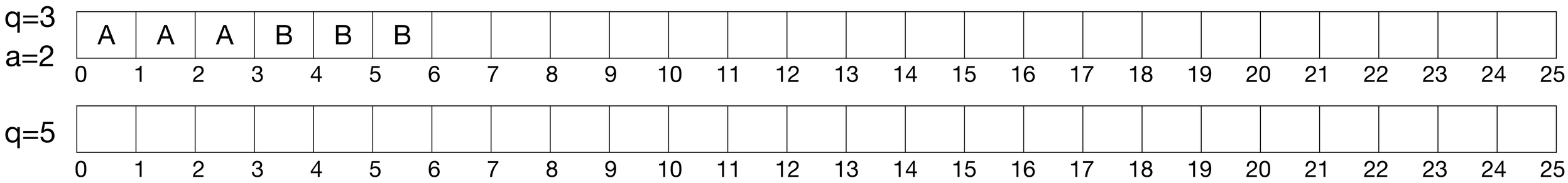


CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

MLFQ when a tie arises, avoid incurring a context switch

processes always start in the top priority queue

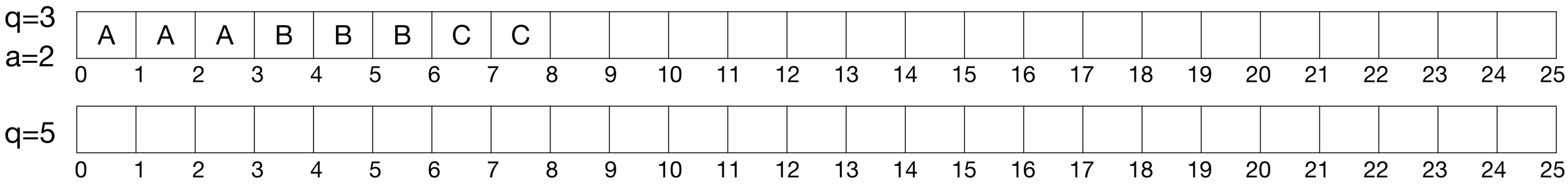


CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

MLFQ when a tie arises, avoid incurring a context switch

the quantum here is 2, so the order is different from the RR(2) example

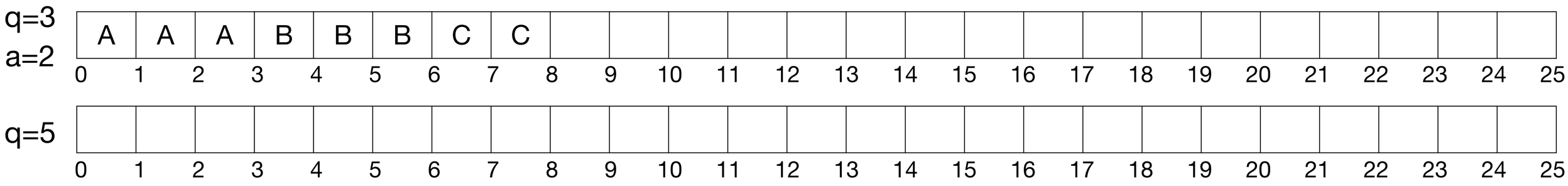


CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

MLFQ when a tie arises, avoid incurring a context switch

C is now done



CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

MLFQ when a tie arises, avoid incurring a context switch

A runs next. D arrives as the A finishes its quantum, but A is re-inserted into the queue first

q=3 a=2	A	A	A	B	B	B	C	C	A	A	A														
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
q=5																									
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

MLFQ when a tie arises, avoid incurring a context switch

q=3
a=2

A	A	A	B	B	B	C	C	A	A	A														

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

q=5

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

MLFQ when a tie arises, avoid incurring a context switch

next is D

q=3
a=2

A	A	A	B	B	B	C	C	A	A	A	D													

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

q=5

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

MLFQ when a tie arises, avoid incurring a context switch

E runs next because it arrived at 4 and B was re-inserted into the queue at 6

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
q=3 a=2	A	A	A	B	B	B	C	C	A	A	A	D	E	E	E											
q=5																										

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

MLFQ when a tie arises, avoid incurring a context switch

B runs again

q=3 a=2	A	A	A	B	B	B	C	C	A	A	A	D	E	E	E	B									
q=5																									

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

MLFQ when a tie arises, avoid incurring a context switch

B is done

q=3 a=2	A	A	A	B	B	B	C	C	A	A	A	D	E	E	E	B										
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
q=5																										
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

MLFQ when a tie arises, avoid incurring a context switch

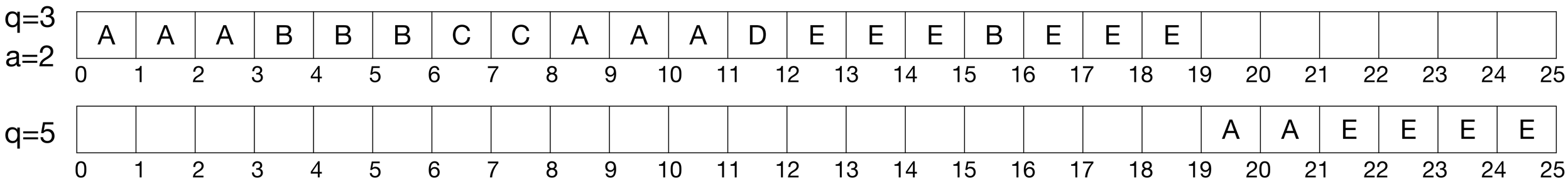
next is E again because A ran out its allotment in this queue and E has not yet

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
q=3 a=2	A	A	A	B	B	B	C	C	A	A	A	D	E	E	E	B	E	E	E							
q=5																										

CPU scheduling: practice

PID	Arrival	CPU Burst
A	0	8
B	1	4
C	2	2
D	3	1
E	4	10

MLFQ when a tie arises, avoid incurring a context switch



A and E finish in the lower queue

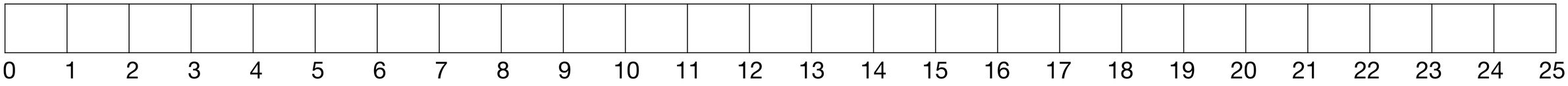
CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	0
B	1	4	2	
C	2	2	2	
D	3	1	1	
E	4	10	4	

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0



the process with the smallest pass value is selected to run

when the process is selected it runs for 1 cycle

then its pass is incremented by its stride

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	4
B	1	4	2	
C	2	2	2	
D	3	1	1	
E	4	10	4	

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A																								

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

A runs

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	4
B	1	4	2	0
C	2	2	2	
D	3	1	1	
E	4	10	4	

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A																								

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

A's pass is incremented by it's stride, so it is 4 now

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	4
B	1	4	2	2
C	2	2	2	
D	3	1	1	
E	4	10	4	

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B																							

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

B runs next, it's pass is incremented by stride=2

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	4
B	1	4	2	2
C	2	2	2	0
D	3	1	1	
E	4	10	4	

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B																							

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	4
B	1	4	2	2
C	2	2	2	2
D	3	1	1	
E	4	10	4	

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C																						

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

C runs next, it's pass is incremented by its stide=2

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	4
B	1	4	2	2
C	2	2	2	2
D	3	1	1	0
E	4	10	4	

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C																						

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	4
B	1	4	2	2
C	2	2	2	2
D	3	1	1	0
E	4	10	4	

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D																					

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	4
B	1	4	2	2
C	2	2	2	2
D	3	1	1	1
E	4	10	4	

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D																					

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	4
B	1	4	2	2
C	2	2	2	2
D	3	1	1	1
E	4	10	4	0

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D																					

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	4
B	1	4	2	2
C	2	2	2	2
D	3	1	1	1
E	4	10	4	4

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D	E																				

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

B and C both have a pass of 2

B ran before C, so we run B again

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	4
B	1	4	2	4
C	2	2	2	2
D	3	1	1	1
E	4	10	4	4

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D	E	B																				

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

B and C both have a pass of 2

B ran before C, so we run B again

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	4
B	1	4	2	4
C	2	2	2	4
D	3	1	1	1
E	4	10	4	4

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D	E	B	C																			

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

C has the smallest pass and runs

Now A, B, and E have the same pass of 4

choose A to run because B and E ran more recently

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	8
B	1	4	2	4
C	2	2	2	4
D	3	1	1	1
E	4	10	4	4

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D	E	B	C	A																		

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

Now A, B, and E have the same pass of 4

choose A to run because B and E ran more recently

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	8
B	1	4	2	4
C	2	2	2	4
D	3	1	1	1
E	4	10	4	8

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D	E	B	C	A	E																

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	8
B	1	4	2	6
C	2	2	2	4
D	3	1	1	1
E	4	10	4	8

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D	E	B	C	A	E	B															

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	8
B	1	4	2	8
C	2	2	2	4
D	3	1	1	1
E	4	10	4	8

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D	E	B	C	A	E	B	B														

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	12
B	1	4	2	8
C	2	2	2	4
D	3	1	1	1
E	4	10	4	8

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D	E	B	C	A	E	B	B	A													

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	12
B	1	4	2	8
C	2	2	2	4
D	3	1	1	1
E	4	10	4	12

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D	E	B	C	A	E	B	B	A	E												

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	16
B	1	4	2	8
C	2	2	2	4
D	3	1	1	1
E	4	10	4	12

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D	E	B	C	A	E	B	B	A	E	A											

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	16
B	1	4	2	8
C	2	2	2	4
D	3	1	1	1
E	4	10	4	16

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D	E	B	C	A	E	B	B	A	E	A	E											

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	20
B	1	4	2	8
C	2	2	2	4
D	3	1	1	1
E	4	10	4	16

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D	E	B	C	A	E	B	B	A	E	A	E	A									

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	20
B	1	4	2	8
C	2	2	2	4
D	3	1	1	1
E	4	10	4	20

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D	E	B	C	A	E	B	B	A	E	A	E	A	E								

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	24
B	1	4	2	8
C	2	2	2	4
D	3	1	1	1
E	4	10	4	20

Stride

When a tie arises, pick the process that didn't run the longest

New process pass=0

A	B	C	D	E	B	C	A	E	B	B	A	E	A	E	A	E	A							

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

CPU scheduling: practice

PID	Arrival	CPU	Stride	Pass
A	0	8	4	24
B	1	4	2	8
C	2	2	2	4
D	3	1	1	1
E	4	10	4	24

Stride

When a tie arises, pick the process that didn't run the longest

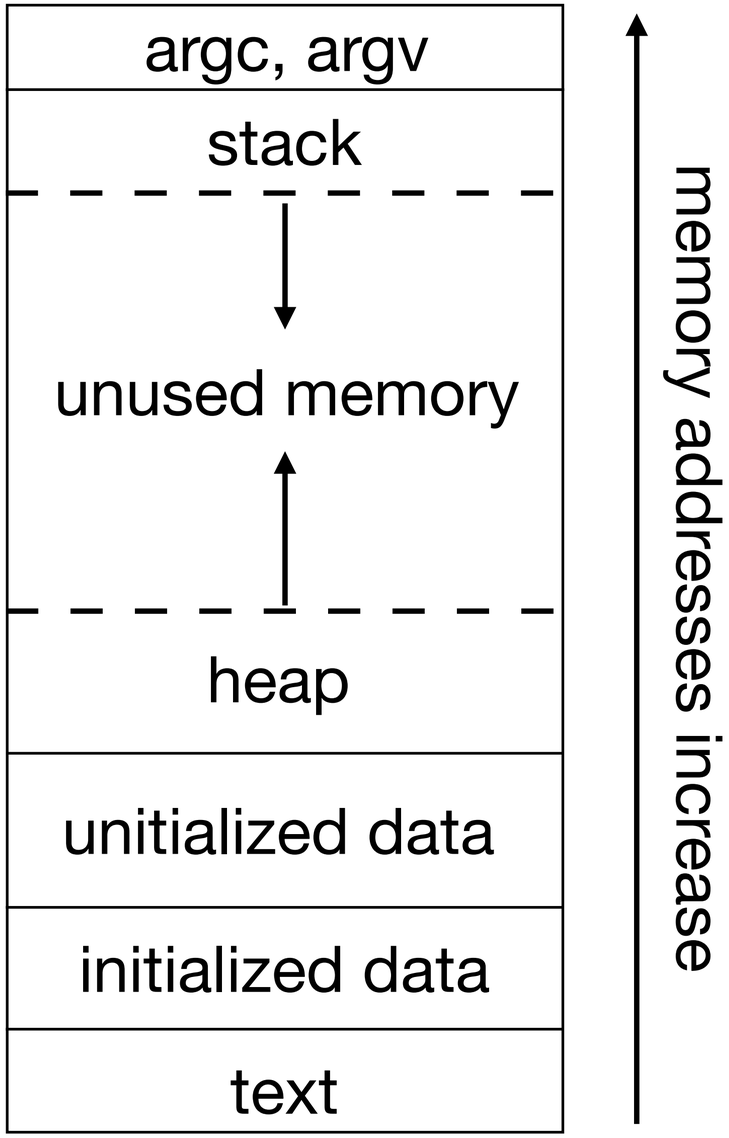
New process pass=0

A	B	C	D	E	B	C	A	E	B	B	A	E	A	E	A	E	A	E	A	E	A	E	E	E

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

Process virtual memory

- **Text** section - executable code
- **Data** section - global variables
- **Heap** section - dynamically allocated memory
- **Stack** section - temp data storage
  - function params
  - local variables
  - return addresses



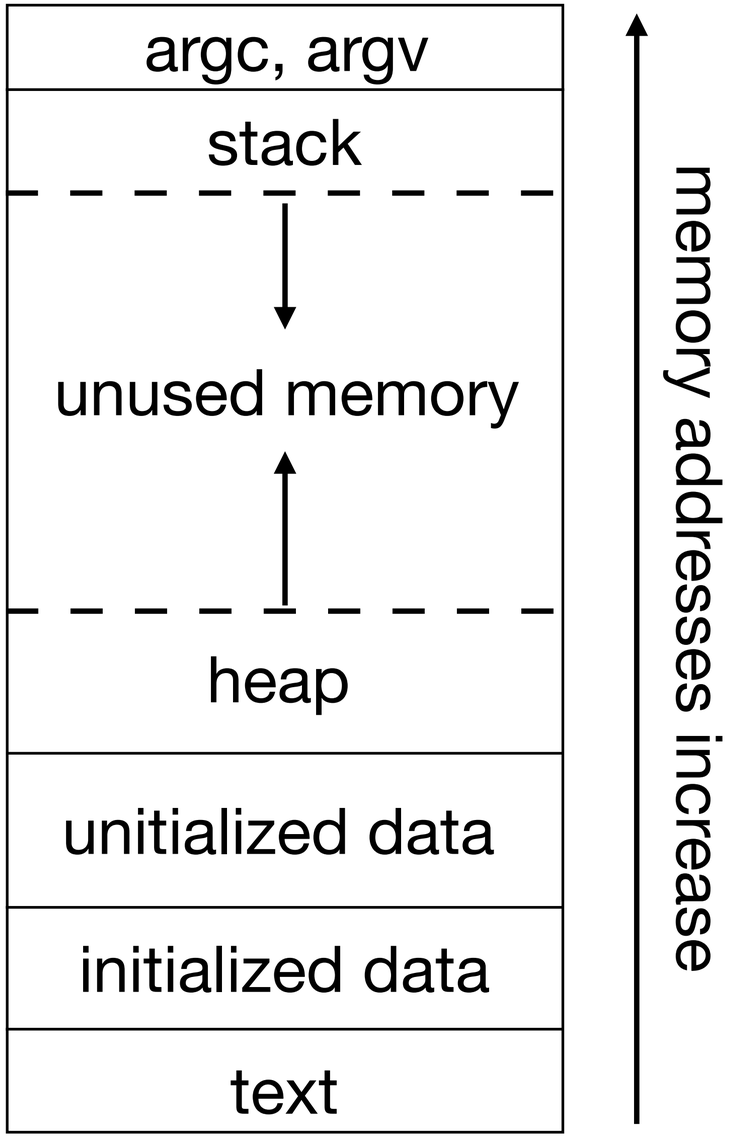
Process virtual memory

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 42;
static int static_global = 99;
int uninit_global;
const char *const_string = "Hi!";

void helper(void) {}

int main(void){
  int local_var = 7;
  static int local_static_var = 123;
  int *var = malloc(10*sizeof(int));
  free(var);
}
```



Process virtual memory

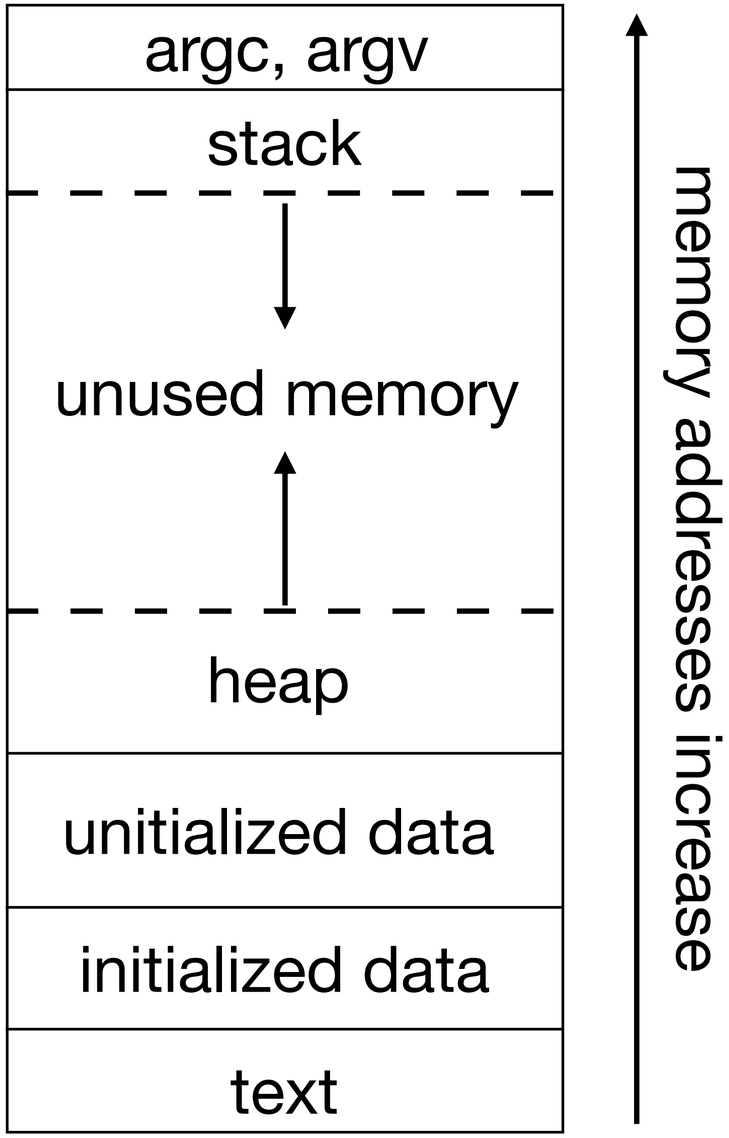
```
#include <stdio.h>
#include <stdlib.h>

int global_var = 42;                    initialized data
static int static_global = 99;          initialized data
int uninit_global;                      unitialized data
const char *const_string = "Hi!";       text

void helper(void) {}                    code

int main(void){
  int local_var = 7;                    stack
  static int local_static_var = 123;    initialized data
stack int *var = malloc(10*sizeof(int)); heap
  free(var);
}
```

static variables (even local ones) persist between calls, so they cannot be on the stack



Memory virtualization

- **Memory virtualization**
  - Processes see their memory as contiguous and virtually unlimited
  - Processes are **not aware** of their memory location in physical memory
  - OS manages **allocation** and **access to memory** to protect processes from each other and itself from processes
- **Address space** - a chunk of memory allocated by the OS for each process
  - processes can allocate/deallocate memory within that chunk
- **Memory API** - `malloc()`, `free()`

Memory allocation

- `malloc()` - allocates
  - a **single contiguous unitialized** block of memory
  - on the **heap**
  - at **runtime**
- `free()` - deallocates
  - only memory **previously allocated by malloc**
  - any other pointer (not from malloc, not on the heap, invalid address) will result in something **very bad** (segfault, bus error, undefined behavior, bugs)
- to `free()` or not to `free()` all malloc'd memory?
  - better to always `free()` unless you know what you are doing

Memory virtualization

- Two related problems
  - Memory allocation: How to find a free block of memory for the process?
  - Address translation: How to translate a virtual address to physical address?

Memory allocation

- Allocate **variable** size blocks - segmentation
  - pros:
    - can allocate memory **separately** for stack/heap/code - reduces **internal fragmentation**
  - cons:
    - **external fragmentation** - allocation and deallocation of variable size blocks leaves "holes" in memory, difficult to reuse it
    - complex address translation - 1 or 3 sets of registers

Memory allocation

- Keep a freelist **embedded** in memory to keep track of empty space
- Strategies may reduce external fragmentation but do not guarantee it
  - **Best fit** - return block closest in size
  - **Worst fit** - return the largest block
  - **First fit** - return the first large enough block
  - **Next fit** - like first fit, but start searching where it was last left off
  - More sophisticated strategies - segregated lists, binary buddy allocation

Paging

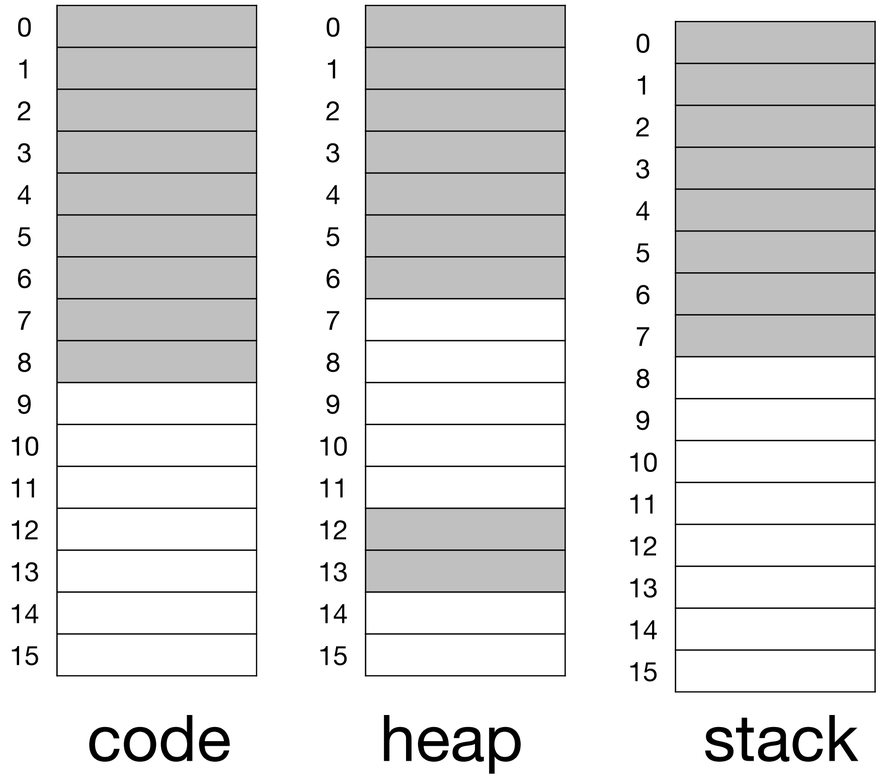
- **Paging** splits memory into **equally-sized** chunks — simpler and more efficient memory allocation
  - Pros
    - fixed-size pages avoid **external fragmentation**
    - **easy to allocate** memory (all blocks are the same, any free one will do)
    - does **not** require hardware support - flexible
  - Cons
    - does not eliminate **internal fragmentation** (if page size is too big)
    - page tables are **large**
    - accessing memory is **slow** (**doubles** the number of memory operations per instruction)

Paging

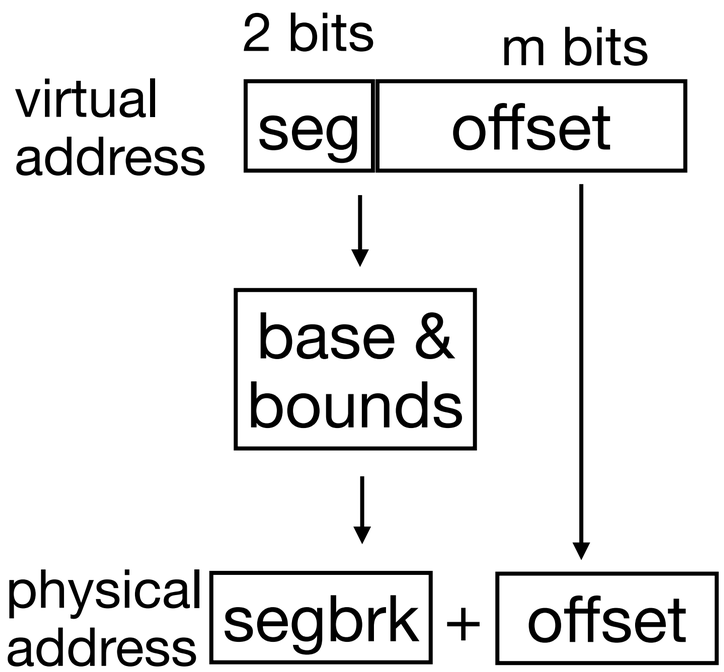
- Problem: How to **reduce memory** required for storing **page tables**?
- Option #1: **larger pages** - reduce page table size, but lead to internal fragmentation
- Option #2: **segmentation for page tables** - no internal fragmentation, but issues with complexity
- Option #3: **multi-level page tables** - turn page table into a tree-like structure
  - less internal fragmentation, but high complexity and slow memory access
- Option #4: **inverted page tables** - index physical pages - one table for all processes
  - save memory, but issues with search and processes sharing memory

Paging + segmentation

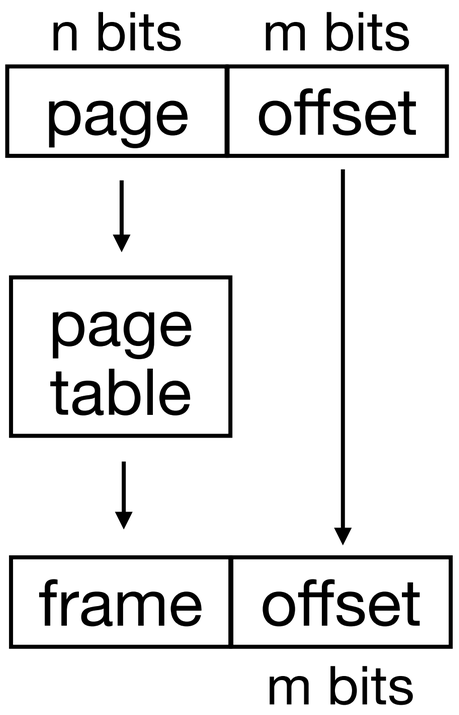
- **Hybridize** paging and segmentation
  - instead of one page table for address space
  - **3 page tables**: for code, heap and stack
  - base register - physical address of the page table for the segment
  - bounds register - number of valid entries in the table for the segment



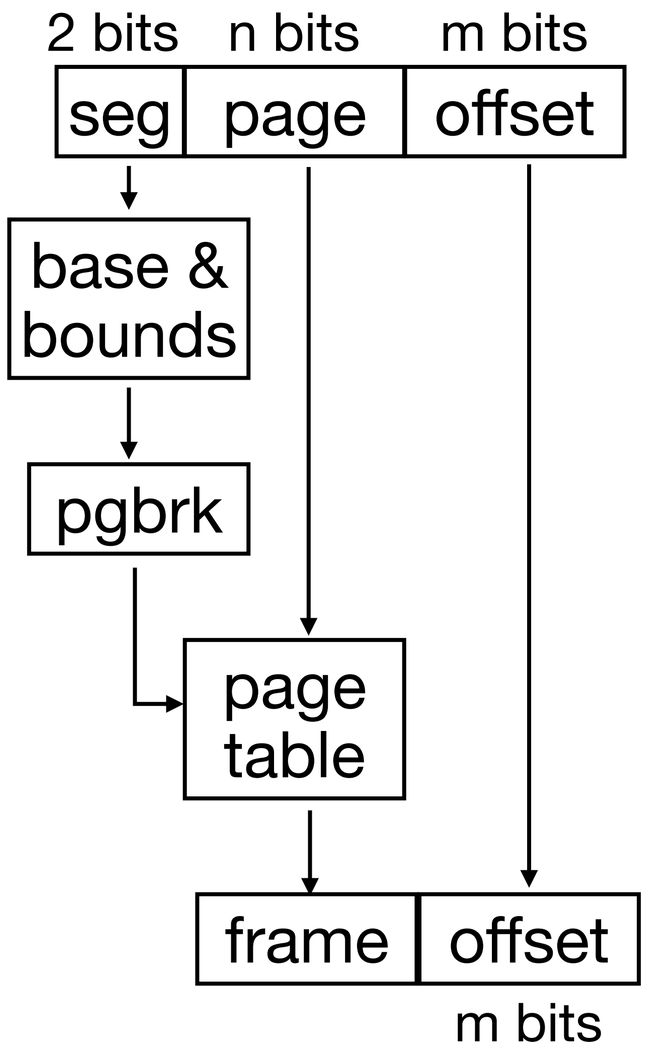
Address translation



segmentation



paging

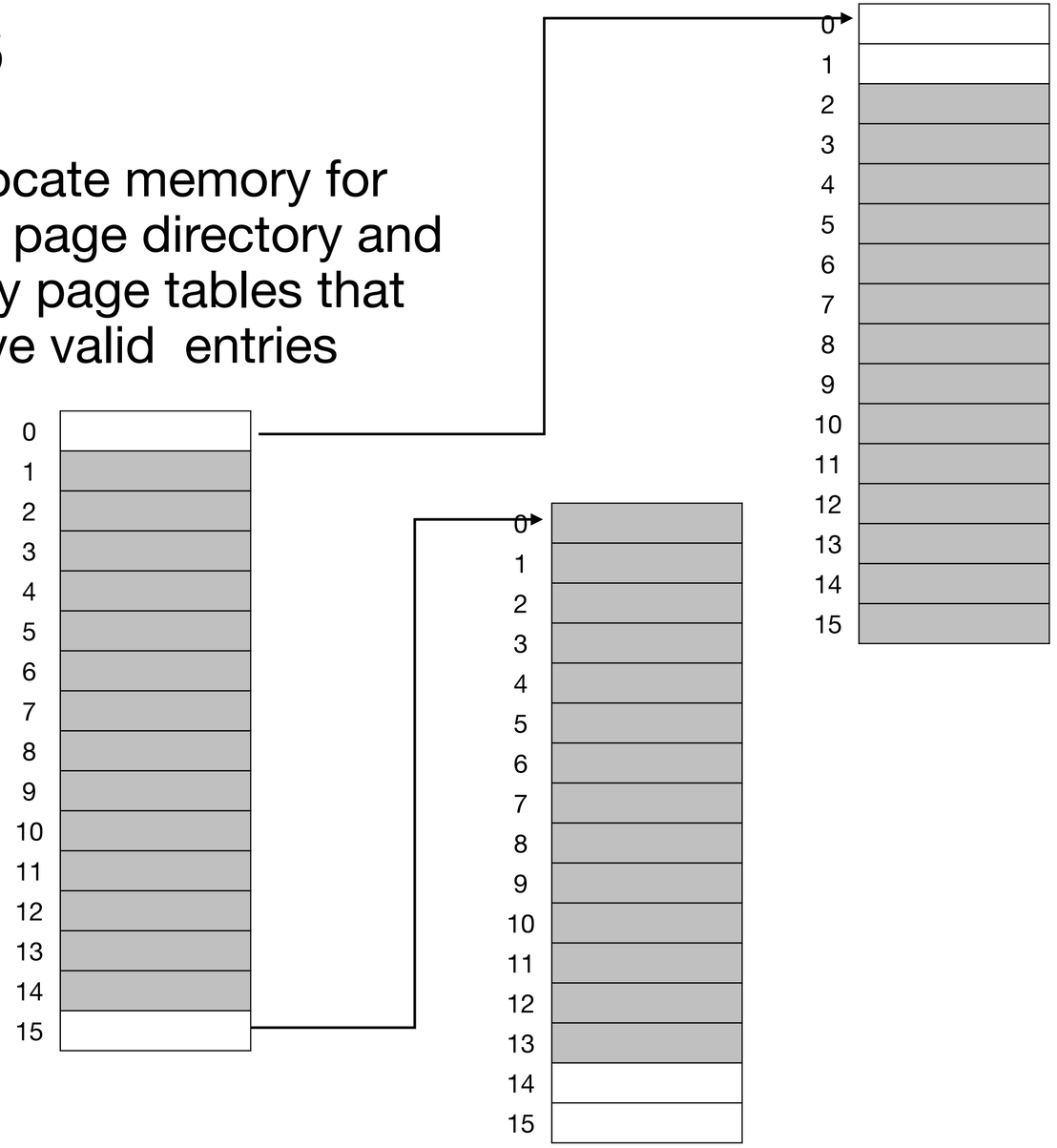


paging + segmentation hybrid

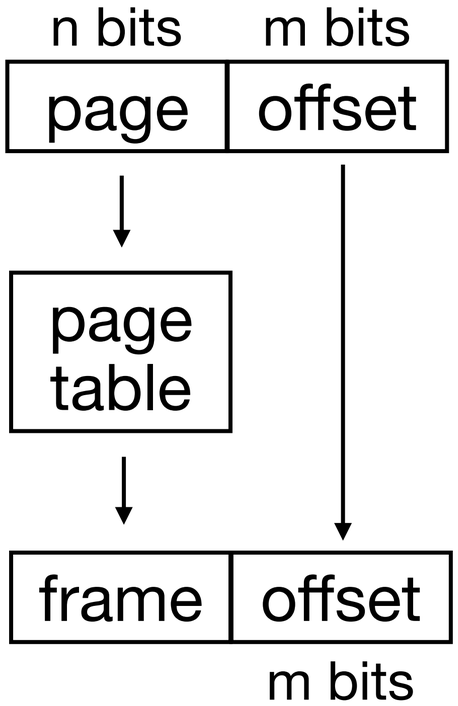
Multi-level page tables

- Make page tables **smaller**
- Allow allocating memory only for used pages
- Good for **sparse** address spaces
- Works with **TLB**
- Cons:
  - memory accesses are **slower**

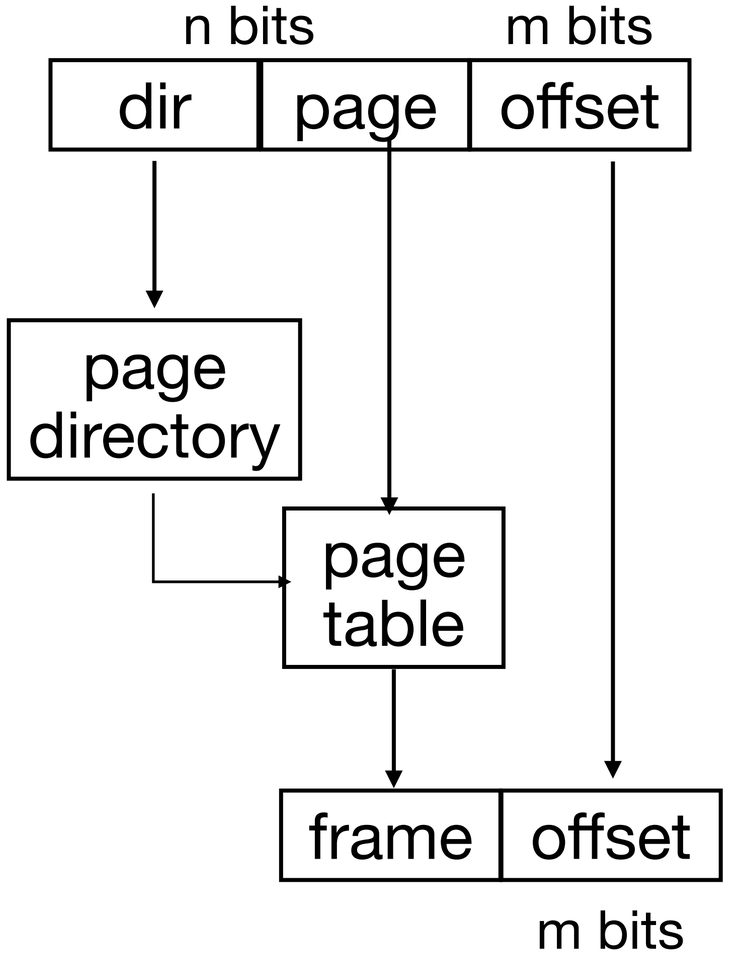
Allocate memory for the page directory and only page tables that have valid entries



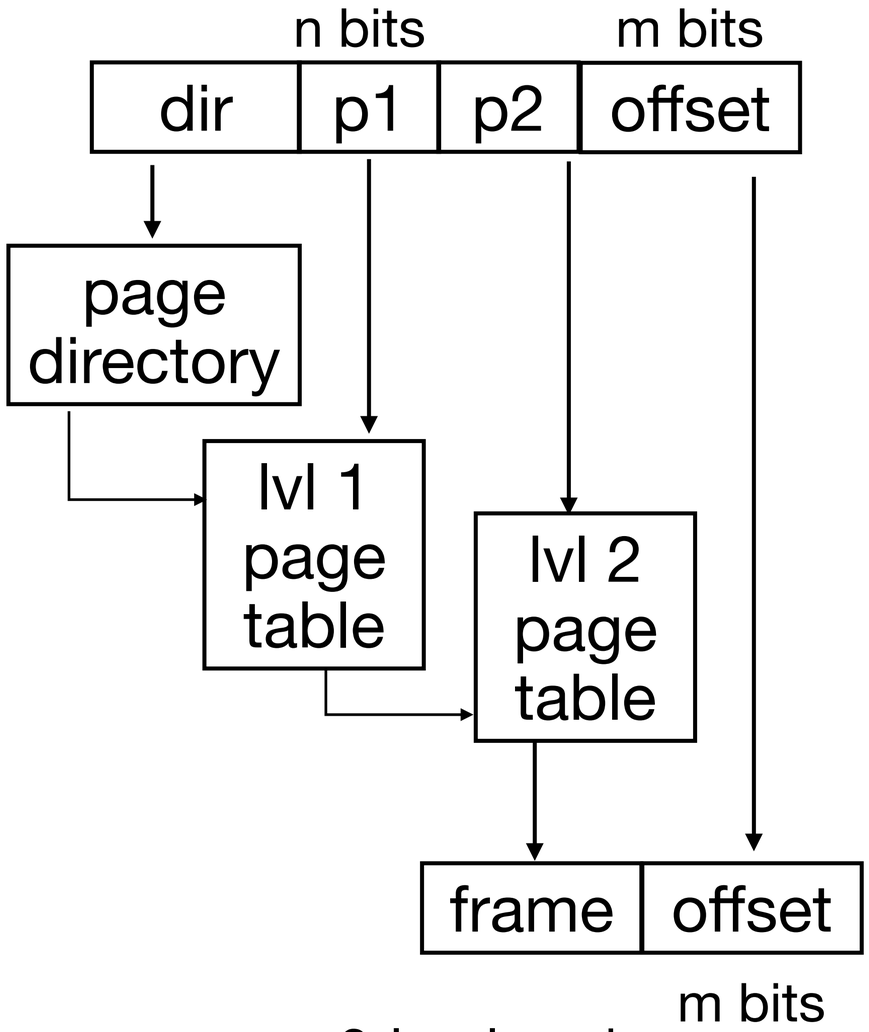
Multi-level page tables



paging

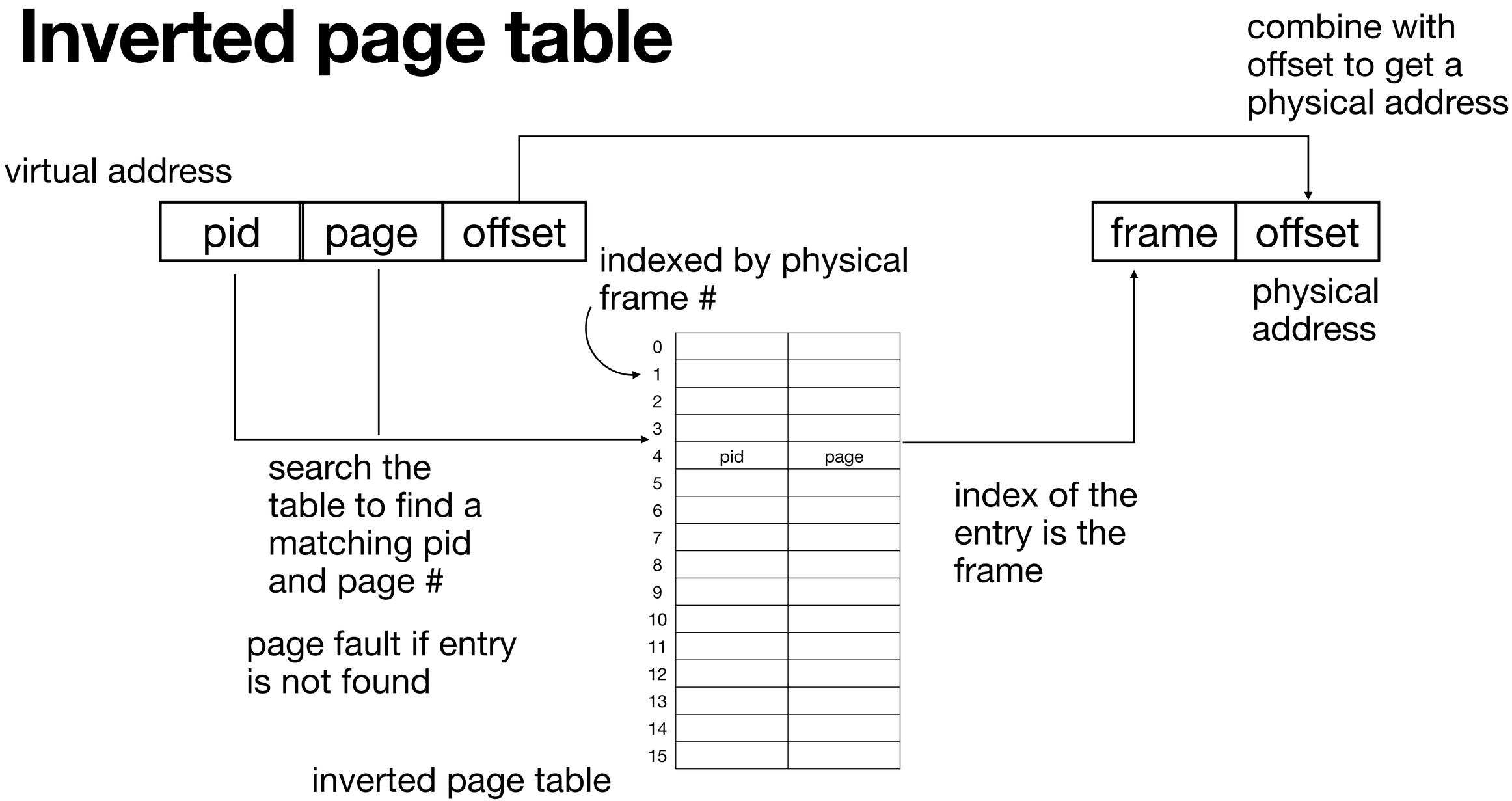


2-level paging



3-level paging

Inverted page table

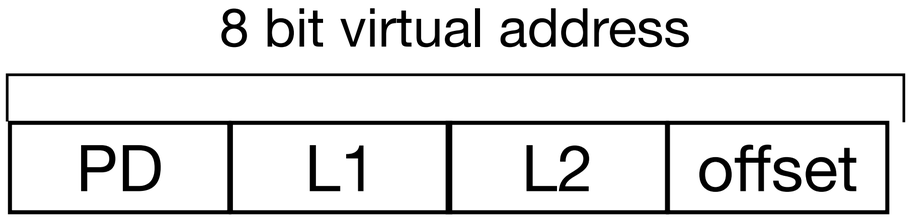


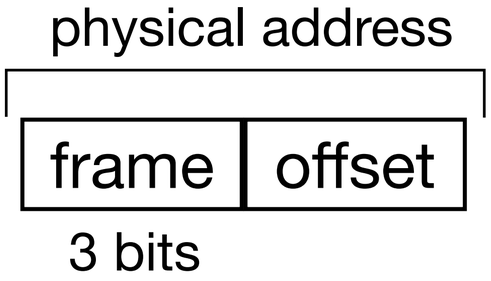
Inverted page table

- **Index physical pages** instead of virtual ones
- Pros:
  - 1 page table for all processes
- Cons:
  - table **is large** - proportional to the size of RAM - search can take **a while**
    - hash table + TLB can improve performance
  - hard to do **shared memory** (multiple pids match to the same physical address)

Paged address

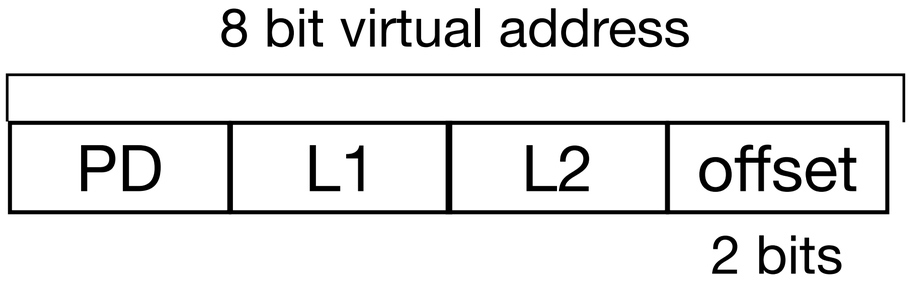
- 8-bit virtual memory address
- 4-byte page size
- 3-bit frame number
- 3-level pages

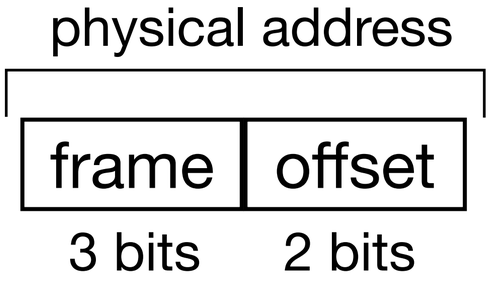




Paged address

- 8-bit virtual memory address
- 4-byte page size -> need 2 bits for offset
- 3-bit frame number
- 3-level pages





Paged address

- 8-bit virtual memory address
- 4-byte page size -> need 2 bits for offset
- 3-bit frame number
- 3-level pages

8 bit virtual address

PD	L1	L2	offset
2 bits	2 bits	2 bits	2 bits

4 entries in PD
4 entries in L1 table
4 entries in L2 table

physical address

frame	offset
3 bits	2 bits

Paged address

- 8-bit virtual memory address
- 4-byte page size -> need 2 bits for offset
- 3-bit frame number
- 3-level pages

8 bit virtual address

PD	L1	L2	offset
2 bits	2 bits	2 bits	2 bits
1 bit	1 bit	4 bits	

2 entries in PD
2 entries in L1 table
16 entries in L2 table

physical address

frame	offset
3 bits	2 bits

Paged address

- 8-bit virtual memory address
- 4-byte page size -> need 2 bits for offset
- 3-bit frame number
- 3-level pages

8 bit virtual address

PD	L1	L2	offset
2 bits	2 bits	2 bits	2 bits
1 bit	1 bit	4 bits	
1 bit	3 bit	2 bits	

…

as long as the bits sum up to 6

2 entries in PD
8 entries in L1 table
4 entries in L2 table

physical address

frame	offset
3 bits	2 bits

Paged address

- 8-bit virtual memory address
- 4-byte page size -> need 2 bits for offset
- 3-bit frame number
- 3-level pages

how many pages in VM?

2 bits needed for offset, this leaves 6 bits for page index, so 2^6=64 pages

how large is VM?
num pages * page size = 64 *4 = 256 bytes

how large is RAM?
num frames * page size = 2^3 * 4 = 32 bytes

8 bit virtual address

PD	L1	L2	offset
2 bits	2 bits	2 bits	2 bits
1 bit	1 bit	4 bits	
1 bit	3 bit	2 bits	
	…		

as long as the bits sum up to 6

2 entries in PD
8 entries in L1 table
4 entries in L2 table

physical address

frame	offset
3 bits	2 bits

Paged address

- 6-bit virtual memory address
- 4-byte page size
- 3-bit frame number
- 2-level pages

2 bits	2 bits	2 bits
PD	L1	offset

virtual address

3 bits	2 bits
frame	offset

physical address

001001 → 01**101**

PD

VPN	Table
00	T1
01	T2
10	- - -
11	T3

T1

VPN	Frame
00	101
01	- - -
10	011
11	110

T2

VPN	Frame
00	100
01	010
10	- - -
11	001

T3

VPN	Frame
00	- - -
01	111
10	000
11	- - -

Paged address

- 6-bit virtual memory address
- 4-byte page size
- 3-bit frame number
- 2-level pages

2 bits	2 bits	2 bits
PD	L1	offset

virtual address

3 bits	2 bits
frame	offset

physical address

100101 → ?

page fault at PD

PD

VPN	Table
00	T1
01	T2
10	- - -
11	T3

T1

VPN	Frame
00	101
01	- - -
10	011
11	110

T2

VPN	Frame
00	100
01	010
10	- - -
11	001

T3

VPN	Frame
00	- - -
01	111
10	000
11	- - -

Paged address

- 6-bit virtual memory address
- 4-byte page size
- 3-bit frame number
- 2-level pages

2 bits	2 bits	2 bits
PD	L1	offset

virtual address

3 bits	2 bits
frame	offset

physical address

110110 → 11110

PD

VPN	Table
00	T1
01	T2
10	- - -
11	T3

T1

VPN	Frame
00	101
01	- - -
10	011
11	110

T2

VPN	Frame
00	100
01	010
10	- - -
11	001

T3

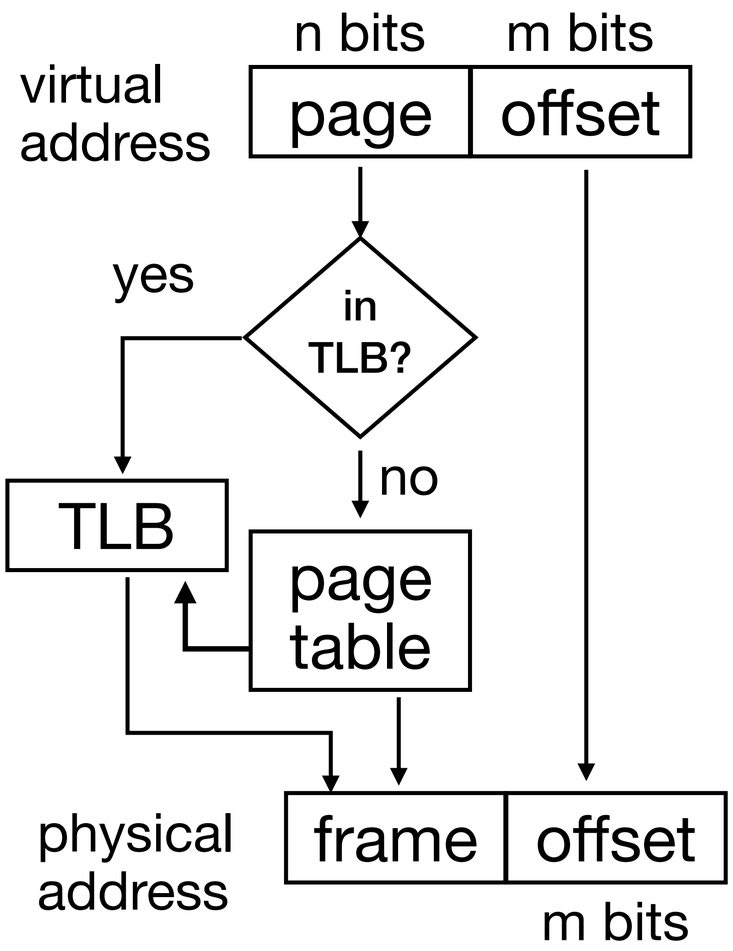
VPN	Frame
00	- - -
01	111
10	000
11	- - -

TLB

- **Translation Lookaside Buffer (TLB)** - memory **cache** for storing translations of virtual memory to physical memory addresses
  - Pros
    - **reduces** memory accesses to page table
    - **very fast** hardware-supported implementation
  - Cons
    - needs **careful management** to avoid too many **TLB misses**

Paging: TLBs

- Problem: **too many memory accesses** needed for paging
- Solution: translation-lookaside buffer (**TLB**)
  - **cache** of virtual-to-physical address translations
  - check if the virtual address is in the TLB
    - if yes, then **avoid page table lookup** altogether
    - if no, perform page table lookup and cache the result

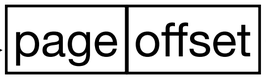


Swapping

- Address space of processes is larger than RAM
- This is the point of virtualization - illusion of infinite space
- **Swapping** helps maintain this illusion
  - Use hard drive to extend the size of RAM

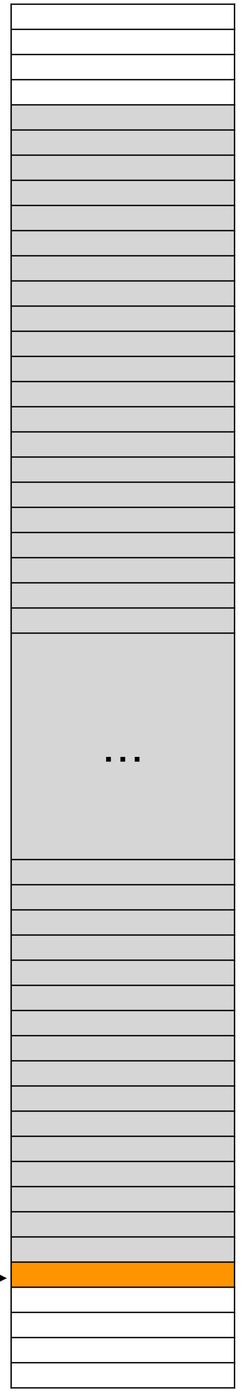
Paging

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
     sum += a[i]
}
```



processes access memory at virtual address

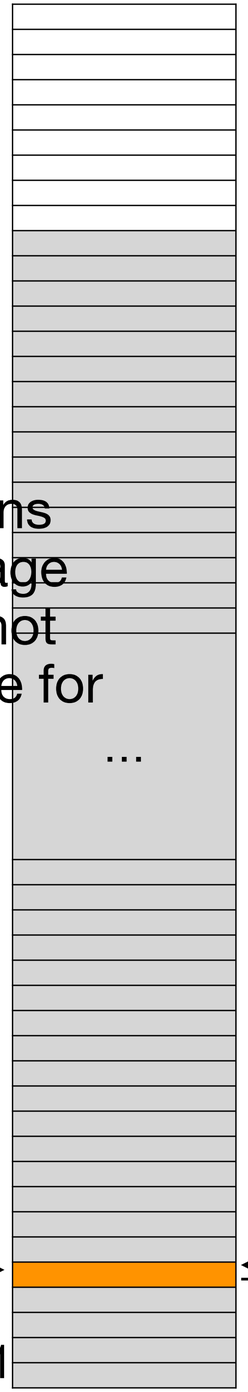
virtual address space



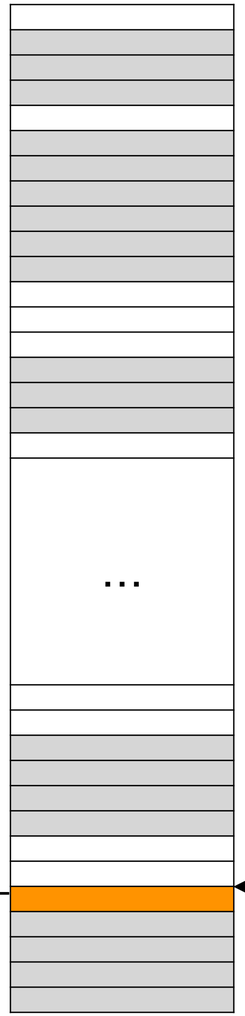
what happens when the page table does not have a frame for our page?

page fault

page table

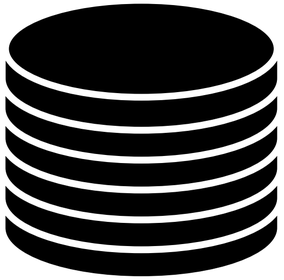


update page table



RAM

load the page into RAM



HD (swap space)

need to get the page from HD

Page table organization

- Assume a linear page table - an array
- Array index is the virtual page number (VPN)

0	frame 3
1	...
...	...
x	frame Y

- Page-table entry (PTE) contains
  - physical frame number (PFN)
  - **valid bit** - marks invalid addresses in the virtual address space (e.g. free space between stack and heap)
  - **present bit** - if this page is in the physical memory
  - **dirty bit** - has this page been modified
  - **reference bit** (aka accessed bit) - has this page been accessed
  - **permissions** - is this address read-only or writable

Cache replacement policy

- Problem: cache is small. Which page should be removed?
  - Option #1: **MIN** - evict the page that will not be used in the future
    - cons: need to predict the future
  - Option #2: **FIFO** - evict the oldest page
    - cons: more likely to throw out heavily used pages
  - Option #3: **Random** - evict a page chosen at random
    - often used as a lower bound on the replacement policy
  - Option #4: **LRU** - evict the page that has not been used the longest
    - a good approximation of MIN

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

Access	Hit/ Miss?	Evict	Cache
0			

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0		

Access	Hit/ Miss?	Evict	Cache
0	Miss		0

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0		

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1			

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	2

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	2

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0			

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	2

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		0 1 2

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	2

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		0 1 2
1			

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	2

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		0 1 2
1	Hit		0 1 2

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	2

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		0 1 2
1	Hit		0 1 2
4			

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	4

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		0 1 2
1	Hit		0 1 2
4	Miss	2	0 1 4

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	4

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		0 1 2
1	Hit		0 1 2
4	Miss	2	0 1 4
0	Hit		0 1 4

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 **4** 1 2 1

0	1	4

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		0 1 2
1	Hit		0 1 2
4	Miss	2	0 1 4
0	Hit		0 1 4
4	Hit		0 1 4

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	4

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		0 1 2
1	Hit		0 1 2
4	Miss	2	0 1 4
0	Hit		0 1 4
4	Hit		0 1 4
1	Hit		0 1 4

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	4

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		0 1 2
1	Hit		0 1 2
4	Miss	2	0 1 4
0	Hit		0 1 4
4	Hit		0 1 4
1	Hit		0 1 4
2			

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	2

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		0 1 2
1	Hit		0 1 2
4	Miss	2	0 1 4
0	Hit		0 1 4
4	Hit		0 1 4
1	Hit		0 1 4
2	Miss	4	0 1 2

MIN

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	2

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		0 1 2
1	Hit		0 1 2
4	Miss	2	0 1 4
0	Hit		0 1 4
4	Hit		0 1 4
1	Hit		0 1 4
2	Miss	4	0 1 2
1	Hit		0 1 2

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	2

LRU

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	2

LRU

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0			

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	2

LRU

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

1	2	0

LRU

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

1	2	0

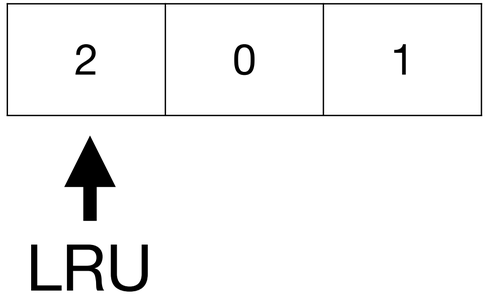
LRU

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0
1			

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1



Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0
1	Hit		2 0 1

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

2	0	1

LRU

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0
1	Hit		2 0 1
4			

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	1	4

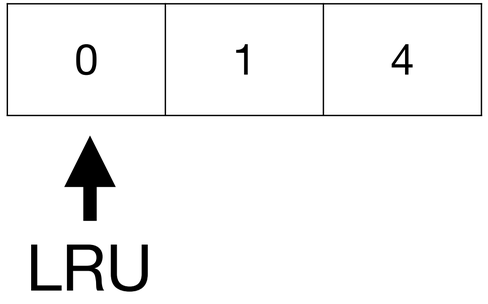
LRU

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0
1	Hit		2 0 1
4	Miss	2	0 1 4

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1



Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0
1	Hit		2 0 1
4	Miss	2	0 1 4
0			

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

1	4	0

LRU

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0
1	Hit		2 0 1
4	Miss	2	0 1 4
0	Hit		1 4 0

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 **4** 1 2 1

1	4	0

LRU

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0
1	Hit		2 0 1
4	Miss	2	0 1 4
0	Hit		1 4 0
4			

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

1	0	4

LRU

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0
1	Hit		2 0 1
4	Miss	2	0 1 4
0	Hit		1 4 0
4	Hit		1 0 4

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

1	0	4

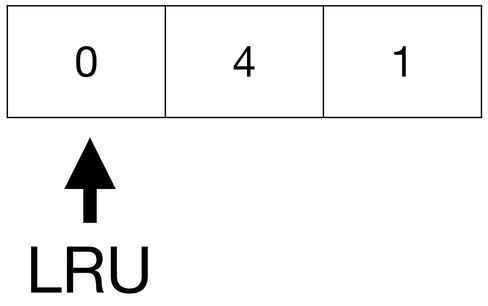
LRU

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0
1	Hit		2 0 1
4	Miss	2	0 1 4
0	Hit		1 4 0
4	Hit		1 0 4
1			

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1



Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0
1	Hit		2 0 1
4	Miss	2	0 1 4
0	Hit		1 4 0
4	Hit		1 0 4
1	Hit		0 4 1

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

0	4	1

LRU

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0
1	Hit		2 0 1
4	Miss	2	0 1 4
0	Hit		1 4 0
4	Hit		1 0 4
1	Hit		0 4 1
2			

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

4	1	2

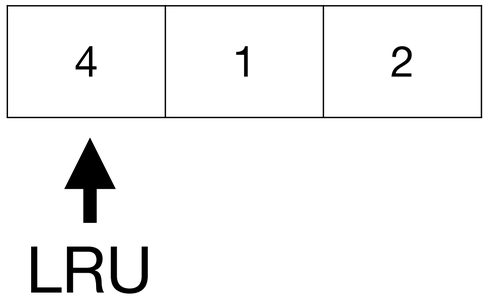
LRU

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0
1	Hit		2 0 1
4	Miss	2	0 1 4
0	Hit		1 4 0
4	Hit		1 0 4
1	Hit		0 4 1
2	Miss	0	4 1 2

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1



Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0
1	Hit		2 0 1
4	Miss	2	0 1 4
0	Hit		1 4 0
4	Hit		1 0 4
1	Hit		0 4 1
2	Miss	1	4 1 2

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1

4	1	2

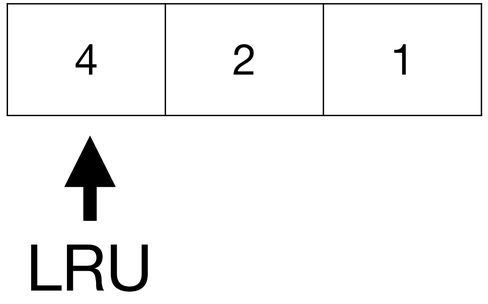
LRU

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0
1	Hit		2 0 1
4	Miss	2	0 1 4
0	Hit		1 4 0
4	Hit		1 0 4
1	Hit		0 4 1
2	Miss	1	4 1 2
1			

LRU

- Assume a cache that fits 3 pages
- Program accesses 4 virtual pages in this order:

0 1 2 0 1 4 0 4 1 2 1



Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0
1	Hit		2 0 1
4	Miss	2	0 1 4
0	Hit		1 4 0
4	Hit		1 0 4
1	Hit		0 4 1
2	Miss	1	4 1 2
1	Hit		4 2 1

0 1 2 0 1 4 0 4 1 2 1

MIN

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		0 1 2
1	Hit		0 1 2
4	Miss	2	0 1 4
0	Hit		0 1 4
4	Hit		0 1 4
1	Hit		0 1 4
2	Miss	4	0 1 2
1	Hit		0 1 2

hit rate

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}} = \frac{6}{6+5} = 54.4\,\%$$

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Noncompulsory Misses}} = \frac{6}{6+1} = 85.7\,\%$$

LRU

Access	Hit/ Miss?	Evict	Cache
0	Miss		0
1	Miss		0 1
2	Miss		0 1 2
0	Hit		1 2 0
1	Hit		2 0 1
4	Miss	2	0 1 4
0	Hit		1 4 0
4	Hit		1 0 4
1	Hit		0 4 1
2	Miss	1	4 1 2
1	Hit		4 2 1

hit rate

$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Misses}} = \frac{5}{6+5} = 45.4\,\%$$

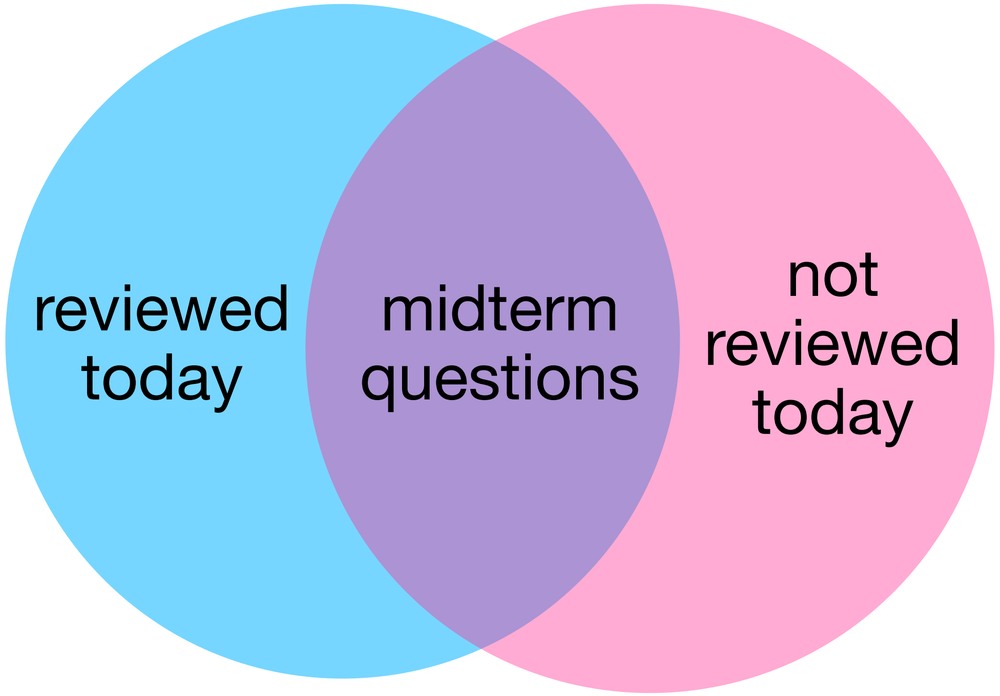
$$\frac{\#\ \text{Hits}}{\#\ \text{Hits} + \#\ \text{Noncompulsory Misses}} = \frac{6}{6+1} = 85.7\,\%$$

Approximating LRU

- Instead of sorting/searching the page table
- Clock algorithm
  - Use the "**referenced bit**" to estimate how recently the page has been used
    - 0=not referenced, 1=referenced
  - Keep processes that have been referenced

Summary

- Not everything reviewed today will be on the midterm
- Some things not reviewed today will be on the midterm



Midterm

- **Feb 26** during class time — **80 minutes**
- In-person, closed book, no calculators
  - **20** multiple choice questions
  - **5** short answer questions
- covers **all** material taught so far
  - lectures 1-11, labs, assignments

A3

- A3 posted yesterday, Feb 23
- Due on **March 16 @ 11:59 EST**
- Covers memory addressing and bitwise operations
- No challenges
  - 90 points for passing the tests + 10 points for readme
- **New** and **improved** autograder 2.0
  - Tracks your **grace days**
  - Catches **200%** more bugs
- Make sure your best submission is the **latest** one

A3

Start early

Quiz 11

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999